

How to Rite: an Engine and a Ritual for Writing Software Engineering Research Papers

Tim Menzies
North Carolina State University
Raleigh, NC, USA
timm@ieee.org

ABSTRACT

Decades of good advice on writing software engineering (SE) research papers [7–9] share one flaw: none of it is executable, so none of it is checked. This paper offers *rite*: a fourteen-step writing loop, eight prompt divisions that encode the advice as machine-readable skills, and a small literature pipeline (search, knee detection, snowballing, keyword coding) over the OpenAlex index [4]. One rule of rite is checkable and surprising: coding 54 papers on multi-objective explanation, only 22 had open-access full text, and their abstracts earned barely half the topic flags that their bodies earned. Abstracts under-report their own papers; so rite requires that a paper’s own title-plus-abstract, coded by its own pipeline, must match the coding of its full text. This paper is written by that method and, per the method, critiques itself. To install, clone github.com/timm/rite and run `etc/init.py`.

1 INTRODUCTION

Everyone who judges a paper cheaply reads only its title and abstract. Rite is a writing loop that makes the cheap read and the deep read agree.

Advice on writing SE research papers is plentiful and good. Shaw taxonomized the questions reviewers ask of ICSE submissions [7]; Widom gave the five questions an introduction must answer [8]; Williams mapped working habits onto empirical careers [9]; community standards now itemize what reviewers may demand [5]. Yet a writer holding all this advice still faces it as prose: nothing connects the checklist to the text in the editor. The advice is never executed, so it is never checked.

Rite makes some of that advice executable. It consists of two small repositories: a read-only *engine* (github.com/timm/rite: a fourteen-step loop, eight prompt divisions, seven Python scripts) and one *workdir* per paper that holds the goal, the coding vocabulary, and everything generated. This paper asks two questions.

RQ1: Can the folklore of paper writing be encoded as a checkable pipeline? Yes, partly. Of the fourteen steps of Table 1, we find **ten** run as scripts or machine-checked gates; the irreducibly human residue is small (steps 10 and 12, plus taste).

RQ2: Is a cheap reading of the literature (title plus abstract) a safe substitute for the full text? No, measurably. In our exhibit, abstracts earned **9 of 21** full-text SE flags, **3 of 10** multi-objective flags, and binary matching over full text saturated on **21 of 22** papers until a frequency threshold was applied (Section 4).

Contributions: (1) the loop and its eight prompt divisions, written so a paper can quote the short form of each; (2) a literature pipeline whose byproducts (knee of the citation curve, download rate, coding grids) become the paper’s own gap argument; (3) the abstract-consistency gate: a submission’s title-plus-abstract must

earn every topic flag its body earns; (4) a replication package: everything at github.com/timm/rite and github.com/timm/how2rite.

1.1 FAQ

Three questions reach us before any others, so we answer them first.

Doesn’t encoding Newell-style problem selection [3] steer a career into a rut? In our experience, no. The literature step keeps forcing contact with material outside prior experience, and the experiments that follow force a critical evaluation of the tool base’s premises. The author offers his own last five years, coded by this same pipeline, in the sequel to this draft; readers can judge the rut for themselves.

Will automation flood the field with average papers? Price’s law observes that the square root of a field’s contributors produce half of its output [2]. The road into that square root is hard thinking, and no pipeline walks it for you. The tools here only cut the ceremony that surrounds the thinking.

Does this kind of tooling ever find anything new? It did here. While exploring this pipeline we found that a paper’s title plus abstract does not code the same way as its full text (Section 4). That hazard has been on record since at least 2007, when Brereton et al. warned that SE abstracts are too poor to rely on when selecting primary studies [1]. But its consequences run far past review screening, since titles and abstracts are what search engines index, what readers use to decide what to read (so miscoding quietly corrupts manual literature reviews), and what bidding systems consult when assigning reviewers to papers. Here, then, is a sizeable problem, barely discussed in SE, uncovered by exactly the kind of LLM-assisted analysis that some researchers would decline to run on methodological or ideological grounds.

1.2 How to install

```
git clone https://github.com/timm/rite
python3 rite/etc/init.py mypaper
cd mypaper
```

The generated *workdir* README needs two lines filled in before anything else runs; for example:

```
# mypaper
goal:  how to write SE research papers
years: 2021–2026
```

Those two lines are the single source of truth for the whole literature search: every downstream script reads its goal and year range from here and nowhere else.

Table 1: The rite loop. Divisions: ch=choose, se=search, en=encode, st=structure, pr=prose, cr=critics, be=bench, sh=ship. Steps marked * are automatic.

#	step	prompts
1	define your area of interest	ch
2	define “recent” (field velocity)	se
3	list your best recent papers	ch
4	* define what you are good at (from 3)	ch
5	* assemble critics (venues, standards, nearest ten)	cr
6	* search: knee, snowball; code full text at threshold	se,en
7	define the most relevant problem (from 6), or the strangest	ch
8	write paper[0]: title, abstract, elevator speech; else goto 7	pr,st
9	i=i+1; write paper[i]; rest stays headers	st,pr
10	offline deep read, 30–60 minutes of comments	cr
11	* apply other critics; split fixes auto/manual	cr
12	discuss with other humans	cr
13	* apply auto fixes; recode own abstract, must match body	en,pr
14	* self-test; fail goto 9, pass ship with replication package	sh

1.3 How to use

Follow Table 1. Steps 1–7 configure and search; step 8 gates entry (no elevator speech, no paper); steps 9–14 loop until the self-test passes. Each step names the prompt division that drives it, and each division is a skill file whose short form fits on one screen.

2 RELATED WORK

This work has excellent ancestors. Shaw’s minitutorial remains the best census of what program committees actually reward, and her three reader questions (what is the contribution, what is the new result, why believe it) are the spine of every section rite generates [7]. Widom’s five-question introduction is the completeness test rite applies to its own openings [8]. Williams connected writing practice to career impact [9], and the SIGSOFT empirical standards turned scattered reviewer folklore into named, checkable expectations [5]. On the analytics side, OpenAlex opened the citation graph that older advice could only gesture at [4], and length-normalized term weighting is textbook information retrieval [6].

None of this, however, executes. The advice papers are prose about prose: no script checks a manuscript against Shaw’s questions, no gate stops a submission whose abstract would be mis-coded by the very search tools that will index it. The standards [5] are checklists for reviewers, applied after writing, not instruments wired into the writing loop itself.

From this survey, three gaps stand out: (1) writing advice is unexecuted and hence unchecked; (2) the literature search that motivates a paper and the coding of that literature are rarely reproducible artifacts of the paper; (3) no standard gate ensures a paper’s abstract matches its body. Accordingly, the rest of this paper describes an engine addressing all three (Section 3), an exhibit measuring issue 3 in the wild (Section 4), and a discussion of what broke (Section 5).

3 THE METHOD

Rite splits into an engine and per-paper workdirs (Figure 1). The engine is read-only at writing time; this is a trust boundary as much

as a convenience, since a tool that edits its own rules while writing paper seven will eventually rot them.

The pipeline is disciplined by two single sources of truth. The search goal and year range live only in the workdir README; the coding vocabulary (topic flags and technology flags, regular-expression families) lives only in the workdir flags.py. Retargeting the whole pipeline to a new paper therefore means editing just two files.

The structure division pins down the shape of the paper itself. Figure 2 states that shape as a definite clause grammar, in the Prolog idiom where each head argument is a key=Variable pair: the keys document which role each part plays, and any part of a parsed paper can be pulled out by name. The keywords earn their keep where one nonterminal is called twice in a single rule (in related, backward and forward snowballing are both just ci tes); there, position alone would be a silent trap.

The pipeline then runs from the workdir: search OpenAlex for the goal; keep the papers above the knee of the sorted-citation curve; backward-snowball their references for the shared classics; fetch open-access PDFs; code abstracts, then code full texts. Full-text coding counts a flag only above a frequency threshold (0.5 matches per 1000 words), a crude form of length-normalized term weighting [6]; the cutoff value is ours and Section 5 owes it a sensitivity analysis.

The loop’s most opinionated gate is the abstract-consistency rule (step 13): run the paper’s own coding grid on its own title plus abstract, and require every flag the body earns at threshold. The justification is Section 4: in the wild, abstracts fail this test about half the time, and everyone who judges a paper cheaply (bidding PC members, systematic-review coders, ranking algorithms, skimming practitioners) reads only the abstract.

4 THE EXHIBIT

We test-drove the pipeline on one goal: explanation of multi-objective reasoning in SE, 2021–2026. The numbers below are the pipeline’s own outputs, kept in the replication package.

Finding 1: the canon is paywalled. Of 54 papers selected for reading (43 recent, above the citation knee; 11 classics, found by snowballing), only 22 (41%) had fetchable open-access PDFs: 47% of the recent, 18% of the classics.

Finding 2: abstracts under-report. Coding the 22 full texts against their abstracts (Table 2): abstracts earned 9 of the 21 SE flags, 3 of the 10 multi-objective flags, 8 of 18 explanation flags. Roughly half the topic signal is invisible to a reader of abstracts alone.

Finding 3: binary matching lies in the other direction. Any-match coding over full text flagged 21 of 22 papers with every flag at once; a ten-page PDF mentions everything somewhere. The frequency threshold restores discrimination, and it discriminates most on the multi-objective flag (11 of 22 papers flip): half this corpus mentions optimization, few work on it. Mentions are not methods.

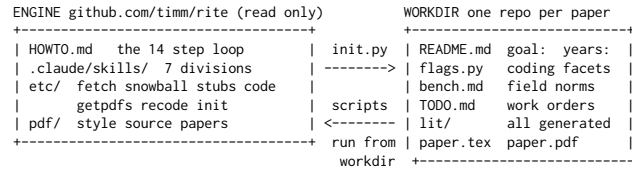


Figure 1: The obligatory system diagram. The argument it walks: advice lives once, on the left, read-only; everything a paper generates lands on the right, so ten papers share one engine and an engine fix reaches all ten. The scripts run from the workdir and read its README and flags; nothing is configured twice.

```

paper(front=F, body=B, back=K) -->
  front(F), body(B), back(K).

front(title=T, author=A, abstract=X) -->
  title(T), author(A), abstract(X).

body(intro=I, related=R, method=M,
  results=E, discussion=D,
  conclusion=C) -->
  intro(I), related(R), method(M),
  results(E), discussion(D),
  conclusion(C).

intro(quote=Q, questions=Qs,
  contributions=Cs, faq=A,
  install=In, use=U) -->
  quote(Q), rqs(Qs), contribs(Cs),
  faq(A), install(In), use(U).

related(before=B, since=S) -->
  cites(B), % backward snowball
  cites(S). % forward snowball

discussion(observations=O, threats=T,
  future=W) -->
  observations(O), threats(T),
  future(W).

back(refs=Rs) --> refs(Rs).

```

Figure 2: The shape of a standard paper, as the structure division states it: a definite clause grammar whose head arguments name their parts.

Table 2: Per-flag coding of 22 papers: abstract versus full text, binary versus thresholded (thr = at least 0.5 matches per 1000 words).

flag	abs	full-bin	full-thr	flips bin→thr
software task	9	22	21	1
multi-objective	3	21	10	11
explanation	8	22	18	4
benchmarking	5	22	20	2

5 DISCUSSION

5.1 Observations

The surprise is not that abstracts compress; it is the asymmetry. Abstracts under-report methodology most (benchmarking flags: 5 in abstracts, 20 in bodies), and full-text binary coding over-reports in exactly the mirror image. Cheap reads and naive deep reads are both wrong, in opposite directions; only the thresholded middle

agrees with a human coder. This is why rite’s gate is stated on the thresholded coding, not on either extreme.

Second, the pipeline’s byproducts kept becoming the argument. The download rate was meant as logging and became Finding 1; the saturation artifact was a bug and became Finding 3. A reproducible search leaves evidence lying around, and evidence is what papers are made of.

Third, the pipeline’s most consequential byproduct may be the problem named in the FAQ (Section 1.1): title-plus-abstract miscoding touches indexing, reading decisions, and reviewer bidding, and SE has barely begun to discuss it [1].

5.2 Threats to validity

Instrument. The 0.5-per-1000-words cutoff is ours, unvalidated beyond this corpus; the promised sensitivity analysis has not yet run. Keyword coding is a draft that the method itself says to hand-audit.

Selection. One goal, one field, 22 full texts; the paywall excluded most classics, so Finding 2 may differ on an older, closed corpus.

Author as instrument. The loop’s expensive steps (10, 12) were run by the person who built the loop. This paper is deliberately pre-critics: it is paper[1] of its own Table 1, awaiting steps 10–13.

5.3 Future work

Run this paper through its own critics (steps 10–12), publish the critic reports and the auto-corrected paper[2] beside this draft; snowball forward and backward from the anchor review to complete the related work; run the threshold sensitivity analysis; add the five-year self-coding exhibit.

6 CONCLUSION

We close by restating the introduction’s claims next to their receipts: the folklore can be partly encoded (ten of fourteen steps run as scripts or gates; the engine and workdir are public); cheap reads are not safe (9 of 21, 3 of 10, 8 of 18 flags visible in abstracts; 21 of 22 papers saturated under binary full-text coding). The method’s one aggressive rule follows from its own measurement: the abstract carries the paper, so gate the abstract. Everything here regenerates from github.com/timm/rite and github.com/timm/how2rite.

REFERENCES

- [1] Pearl Brereton, Barbara A. Kitchenham, David Budgen, Mark Turner, and Mohamed Khalil. 2007. Lessons from Applying the Systematic Literature Review Process within the Software Engineering Domain. *Journal of Systems and Software* 80, 4 (2007), 571–583. <https://doi.org/10.1016/j.jss.2006.07.009>
- [2] Derek J. de Solla Price. 1963. *Little Science, Big Science*. Columbia University Press, New York.

- [3] Allen Newell and Herbert A. Simon. 1976. Computer Science as Empirical Inquiry: Symbols and Search. *Commun. ACM* 19, 3 (1976), 113–126.
- [4] Jason Priem, Heather Piwowar, and Richard Orr. 2022. OpenAlex: A Fully-Open Index of Scholarly Works, Authors, Venues, Institutions, and Concepts. arXiv:2205.01833.
- [5] Paul Ralph et al. 2020. Empirical Standards for Software Engineering Research. arXiv:2010.03525.
- [6] Gerard Salton and Christopher Buckley. 1988. Term-Weighting Approaches in Automatic Text Retrieval. *Information Processing & Management* 24, 5 (1988), 513–523.
- [7] Mary Shaw. 2003. Writing Good Software Engineering Research Papers. In *Proceedings of the 25th International Conference on Software Engineering (ICSE 2003)*. IEEE, 726–736.
- [8] Jennifer Widom. 2006. Tips for Writing Technical Papers. Stanford University. <https://cs.stanford.edu/people/widom/paper-writing.html>, accessed July 2026.
- [9] Laurie Williams. 2011. Seven Habits of Highly Impactful Empirical Software Engineers. Keynote, 7th International Conference on Predictive Models in Software Engineering (PROMISE), Banff, Canada.